

Egyptian E-Learning University
Faculty of Computers & Information Technology

NileGuide

Tourism Planning & Recommendation Platform

Final Graduation Project Documentation

Submitted by

Team Member	ID
Yossef Abdelmoneim Abdel hamid	2200395
Abdelrahman Tamer Mahmoud	2200406
Mohamed Ahmed Attia	2200401
Youssef Mohamed Ali	2200718
Ziad hamdy mohamed	2202656
Zyad Samy Mohamed	2200399
Ahmed Hassan Younes	2202660
Ahmed Hesham Abdelshakour	2200783

Table 1. NileGuide team members

Supervised by

Dr. Kamal Hamza

Assistant: Eng. Mohamed Kamal

Academic Year 2025-2026

Abstract

NileGuide is a web-based tourism planning platform designed to help tourists discover Egyptian activities, compare options, save interesting places, build a personal trip schedule, and ask questions through an AI knowledge assistant. The delivered system includes a tourist interface and an administrative dashboard. The tourist side covers activity discovery, dynamic details, wishlist, schedule, map, profile, contact, authentication and chatbot features. The admin side covers user management, activity management, reporting and AI file management. The final implementation uses Angular Standalone on the frontend, ASP.NET Core 8 Web API on the backend, SQL Server through Entity Framework Core for relational storage, Cloudinary for images, and a separate FastAPI RAG service using MongoDB and Qdrant for document-based AI answers. This document describes the completed implementation, the final system architecture, the database design, the APIs, the frontend screens, the AI/RAG flow, and the testing performed. It also explains what changed from the first-term plan: the AI component moved from a planned recommendation concept to a working knowledge-base assistant, while booking/payment and refresh-token or Remember Me user flows remain future work rather than delivered features.

Document Style and Scope

This final report is written as a complete project documentation file, not only as a presentation summary. It explains the real implementation built by the team and avoids documenting features that were only planned but not delivered. Screenshots are placed before their explanation so the reader can first see the screen and then understand the user action, the purpose of the feature and the connected system behavior.

Table of Contents

- Chapter 1 - Introduction
- Chapter 2 - Background and Related Work
- Chapter 3 - Requirements and System Analysis
- Chapter 4 - System Design
- Chapter 5 - Backend Implementation
- Chapter 6 - Frontend Implementation
- Chapter 7 - AI/RAG Implementation
- Chapter 8 - Graphical User Interface and Feature Walkthrough
- Chapter 9 - Testing and Validation
- Chapter 10 - Conclusion and Future Work
- References
- Appendices

Chapter 1 - Introduction

1.1 Project Overview

NileGuide is a tourism planning and recommendation platform focused on Egyptian travel experiences. The system gives tourists a single place to explore activities, view details, save options, create a schedule and locate planned activities on a map. It also gives administrators a dashboard for controlling platform content and reviewing activity statistics.

The project was initially introduced during the first term as a platform for activity discovery, planning and AI-assisted recommendations. During implementation, the core planning features were completed, and the AI idea was narrowed into a practical RAG assistant that can answer from uploaded tourism documents. This made the final product more realistic and easier to maintain.

1.2 Motivation

- Tourists often collect travel information from social media, blogs, booking sites and informal recommendations. This creates duplicated effort and inconsistent decisions.
- A tourist may like an activity but still need a separate way to save it, compare it with others and fit it into a trip plan.
- Administrators need a controlled way to add, update and remove activities instead of changing static pages or hard-coded data.
- AI answers should be grounded in trusted project documents and tourism knowledge rather than generic responses.

1.3 Problem Definition

The main problem addressed by NileGuide is the lack of a unified planning experience for Egyptian activities. Existing online sources may show attractions, but they do not always combine discovery, details, wishlist, schedule, map visualization, profile preferences, administration and AI knowledge in one coherent system.

Problem area	How it appears
Scattered information	Activities, prices, locations and practical notes are usually distributed across many websites.
Weak planning continuity	Users can discover places but may not have a structured way to save and schedule them.
Content maintenance	A tourism platform needs admin tools for keeping activity data, images and status updated.
Knowledge updates	AI answers should be updated from uploaded files without changing code every time.

Table 2. Problem definition summary

1.4 Project Objectives

- Build a responsive web platform that supports tourist and admin roles.
- Provide a dynamic activity catalogue with search, category and city filters.
- Implement activity details pages that load by route id and backend data.
- Allow tourists to save activities to a wishlist and add them to a trip schedule.
- Display scheduled activities on a map to support location-aware planning.
- Provide profile management including personal data, preferences and profile picture.
- Provide admin tools for managing users, activities, reports and AI files.
- Implement a RAG-based AI assistant that answers using uploaded knowledge documents.

1.5 Scope of the Delivered System

Scope category	Description
Delivered	Authentication, activities, details, wishlist, schedule, map, profile,

	admin dashboard, reports, AI file panel, RAG chatbot.
Not delivered as final features	Payment, real booking confirmation, advanced recommendation engine, Remember Me UX, refresh-token UX.
Future extensions	Personalized trip recommendations, Arabic UI, richer reports, schedule optimization and provider booking integration.

Table 3. Final project scope

Chapter 2 - Background and Related Work

2.1 Tourism Planning Systems

Tourism planning systems usually try to reduce the effort required to create an itinerary. Some systems focus on automatic route generation, while others focus on recommendations using user preferences, ratings or social media behavior. NileGuide combines discovery and planning but keeps the first delivered version practical: the user chooses activities manually, while AI helps with knowledge-based questions rather than making unsupported decisions.

Related direction	Main idea	Gap addressed by NileGuide
ATIPS / itinerary planning	Automatically constructs domestic travel itineraries using popularity, cost, distance and time.	Useful for optimization but usually requires accurate travel-time and schedule constraints.
Smart tourism recommendation systems	Use collaborative/content-based filtering or multi-agent systems to recommend packages.	Often evaluated on limited datasets and may not include admin operations.
Photo/social-based recommenders	Infer interests from public social media images or labels.	Raises privacy concerns and may not represent actual tourist intent.
Travel web aggregators	Combine hotels, flights, packages and chatbot interfaces.	May depend on scraping and external providers, which can be unstable.
NileGuide approach	Combines activity catalogue, planning tools, admin management and RAG assistant.	Focuses on a deployable graduation system with clear delivered modules.

Table 4. Related work and research gap

2.2 Research Gap

- Many systems recommend places but do not provide a full management dashboard for maintaining the catalogue.
- Some systems generate itineraries but do not give the tourist control over wishlist and schedule decisions.
- Chatbot systems may answer generally, while NileGuide uses a RAG knowledge base that the admin can update.
- Operational concerns such as role access, image management, reports and deployment are frequently under-specified in prototype systems.

Chapter 3 - Requirements and System Analysis

3.1 Stakeholders

Stakeholder	Role in the system
Tourist / User	Registers, logs in, discovers activities, saves wishlist items, plans schedule, views map and asks chatbot questions.
Administrator	Manages users, activity content, status, reports and AI knowledge files.
Project team	Maintains frontend, backend, database, AI/RAG service and deployment.
Supervisor / evaluator	Reviews the architecture, implementation, documentation and final demo.

Table 5. Stakeholders

3.2 Functional Requirements

ID	Requirement	Final status
FR-01	User registration and login	Implemented through Auth APIs and Angular auth pages.
FR-02	Role-based access	Tourist and Admin roles control route access and API authorization.
FR-03	Browse activities	Activities page loads dynamic data and supports filters and sort.
FR-04	View activity details	Dynamic /activities/:id route displays details, gallery, documents, map and reviews.
FR-05	Wishlist	Tourist can save, view and remove activities.
FR-06	Schedule	Tourist can add planned activities and export/inspect itinerary.
FR-07	Map	Planned activities are visualized using map markers and itinerary panel.
FR-08	Profile	Tourist can edit profile data, travel dates, preferences and picture.
FR-09	Admin users	Admin can view, search, filter and manage users.
FR-10	Admin activities	Admin can view, search, filter, add, update and delete activities.
FR-11	Reports	Admin can view platform metrics and charts.
FR-12	AI/RAG	Admin uploads knowledge files and chatbot answers from indexed documents.

Table 6. Functional requirements

3.3 Non-Functional Requirements

Quality attribute	Implementation consideration
Usability	Simple navigation, cards, filters, modals, readable dark UI and responsive layout.
Security	JWT Bearer authentication, role authorization and protected admin routes.
Maintainability	Frontend services, backend controllers/services/DTOs, separate AI service.
Scalability	Frontend, backend and AI can be deployed independently.
Reliability	API validation, error handling, loading states and reportable screenshot testing.
Extensibility	Activities, categories, cities, documents and AI knowledge can be expanded.

Table 7. Non-functional requirements

3.4 Main User Stories

- As a tourist, I want to browse activities by city and category so that I can quickly find suitable activities.
- As a tourist, I want to view activity details before deciding whether to save or schedule the activity.
- As a tourist, I want to save activities to a wishlist so that I can compare them later.
- As a tourist, I want to add activities to a schedule so that I can build an itinerary.

- As an admin, I want to manage activities so that the public catalogue remains accurate.
- As an admin, I want to upload AI knowledge files so that chatbot answers stay updated without code changes.

Chapter 4 - System Design

4.1 High-Level Architecture

The final system is divided into three main runtime parts: the Angular frontend, the ASP.NET Core backend and the FastAPI RAG service. The frontend communicates with the .NET backend for core platform features and with the AI service for knowledge-base chat and file management. SQL Server stores the relational platform data, while MongoDB and Qdrant support the AI document-processing workflow.

NileGuide System Architecture

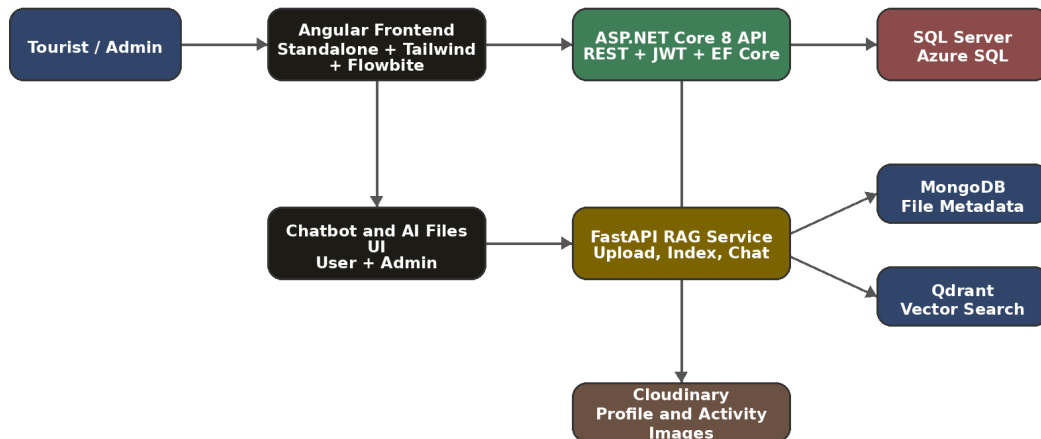


Figure 1. NileGuide final system architecture

4.2 User Workflow

Tourist Workflow in the Delivered System

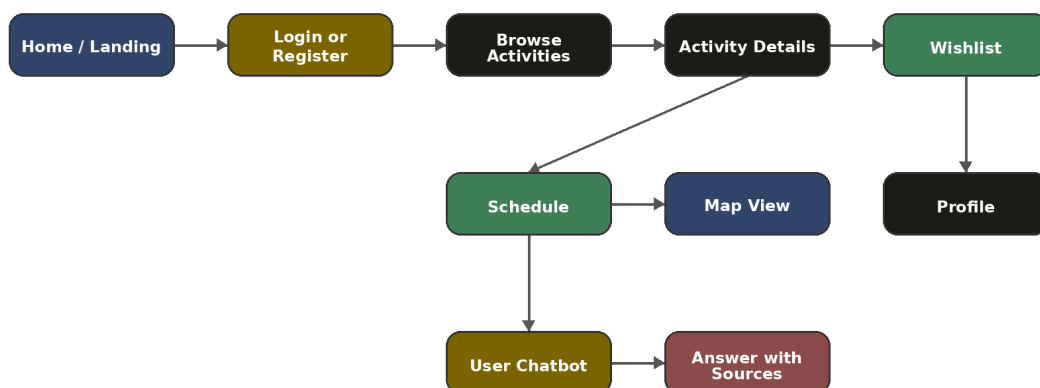


Figure 2. Tourist workflow from discovery to planning and chatbot usage

The tourist workflow starts from the landing page, continues through authentication and activity discovery, and then branches into details, wishlist, schedule, map, profile and chatbot features. The workflow is intentionally simple: the system helps the tourist make decisions without forcing an automatic plan.

4.3 Admin Workflow

Admin Workflow

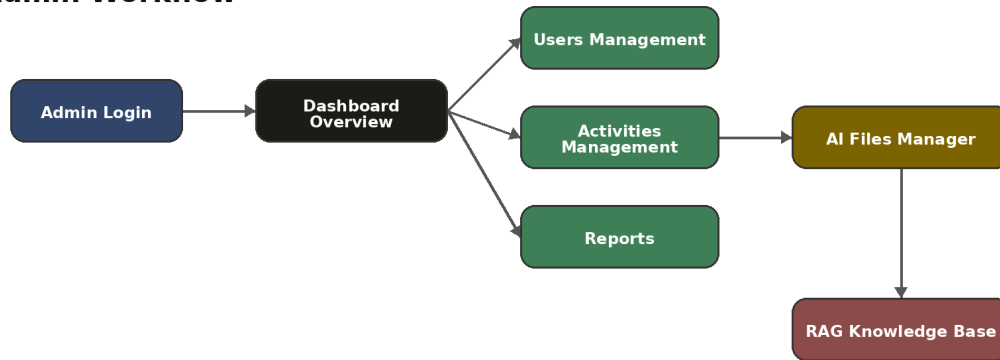
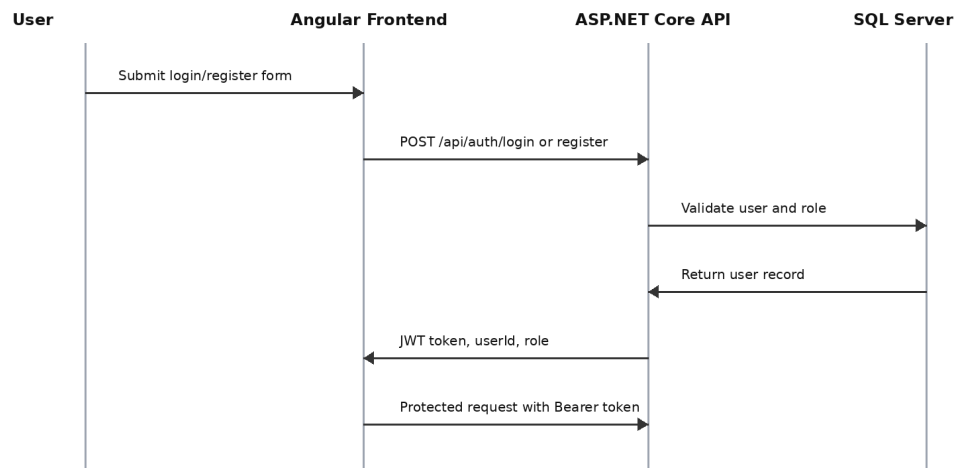


Figure 3. Admin workflow for operational modules

The admin workflow is separated from tourist pages. After login, the administrator accesses a dashboard shell and navigates to users, activities, reports and AI file management. This separation reduces accidental access and makes the demo clearer during discussion.

4.4 Authentication Design

Authentication and Authorization Sequence



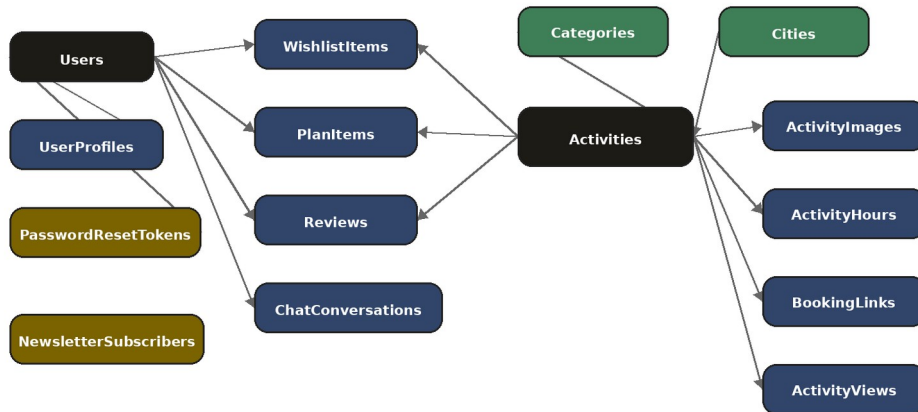
Final report scope: JWT access token and role-based route protection. Remember Me and refresh-token UX are not treated as

Figure 4. Authentication and authorization sequence

Authentication is based on JWT access tokens and role-based access. The frontend stores the access token and sends it in the Authorization header. The final documentation does not describe Remember Me or refresh-token UX as delivered features, because those flows were not part of the final confirmed scope.

4.5 Database Design

Core Database Design



The diagram shows the relationships used by discovery, profile, wishlist, schedule, review, reporting, and chat modules. Refresh tokens exist in k

Figure 5. Simplified database relationship diagram

The relational database stores users, profiles, activities, categories, cities, images, opening hours, booking links, wishlist items, plan items, reviews, activity views, newsletter subscribers and chat conversations. The design supports both the public tourist pages and the admin dashboard.

Entity	Purpose
Users	Authentication, role and account data.
UserProfiles	User personal data, nationality, travel dates, preferred cities and categories.
Activities	Core tourism activity information.
Categories / Cities	Lookup data used by filters and details.
ActivityImages / ActivityHours / BookingLinks	Supporting activity media, times and providers.
WishlistItems	Saved activities for a tourist.
PlanItems	Scheduled activities with selected date and time.
Reviews	User rating and review records.
ActivityViews	Report data for activity-view analytics.
ChatConversations	Saved chat-conversation metadata used by chatbot history.

Table 8. Main database entities

4.6 Deployment Design

Deployment View

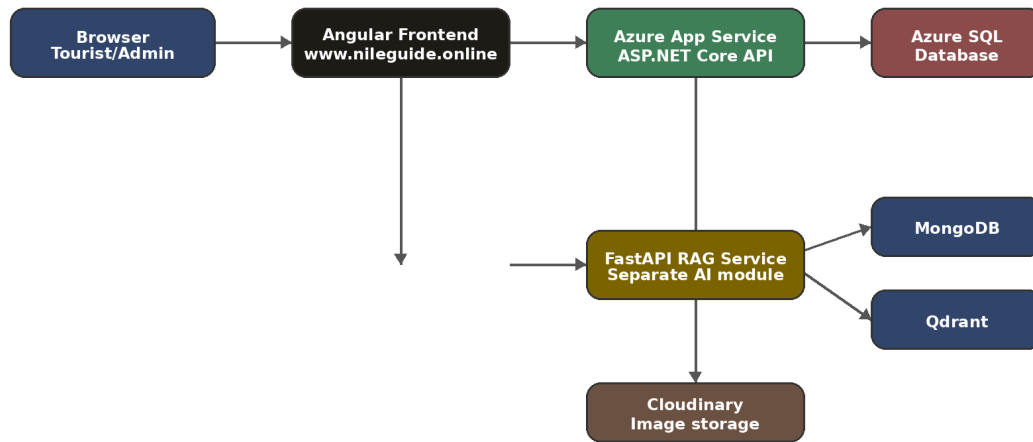


Figure 6. Deployment view for frontend, backend and AI services

The frontend can be hosted on the public NileGuide domain, while the ASP.NET Core API is deployed separately on Azure App Service and connected to Azure SQL. The AI service is separate because it requires different dependencies and storage patterns such as MongoDB and Qdrant. Images are stored through Cloudinary rather than being kept directly in the relational database.

Chapter 5 - Backend Implementation

5.1 Backend Structure

The backend is implemented using ASP.NET Core 8 Web API. It follows a practical layered style: controllers receive HTTP requests, DTOs define request and response shapes, services contain business logic, EF Core models represent database entities, and ApplicationDbContext maps relationships to SQL Server. This structure keeps the implementation organized and easier to test or extend.

Backend part	Implementation detail
Controllers	ActivitiesController, AdminActivitiesController, AuthController, UsersController, UserProfilesController, WishlistController, PlanController, ReviewsController, ReportsController, DashboardController and lookup controllers.
DTOs	Separate request/response models for activities, auth, dashboard, profile, plan, reports, reviews, wishlist and user management.
Services	ActivityService, AdminActivityService, AuthTokenService, UserProfileService, PlanService, WishlistService, ReviewService, DashboardService and ReportService.
Data layer	AppDbContext, EF Core entity configuration and SQL Server connection.
Options	Cloudinary, JWT, Email, Security and connection-string settings.
Middleware/Hubs	Exception handling middleware and SignalR dashboard hub.

Table 9. Backend code organization

5.2 Backend Services and Responsibilities

- AuthTokenService creates the JWT access token used by the frontend after login or registration.
- ActivityService handles public activity retrieval, filtering and mapping into card/details DTOs.
- AdminActivityService handles create, update and delete operations for activities, including image-related data.
- WishlistService and PlanService connect tourist actions to persisted wishlist and schedule records.
- UserProfileService manages profile fields and selected preferences.
- ReportService and DashboardService prepare the metrics used in the admin reports and dashboard cards.
- CloudinaryProfilePictureService and ActivityImageService move image storage outside the database and store URLs in SQL Server.

5.3 API Modules

Module	Endpoints	Purpose
Auth	POST /api/auth/register; POST /api/auth/login; POST /api/auth/forgot-password; POST /api/auth/verify-reset-code; POST /api/auth/reset-password	Account creation, login and password reset.
Activities	GET /api/Activities; GET /api/Activities/{id}	Public catalogue and details.
Reviews	GET /api/activities/{activityId}/reviews; POST /api/activities/{activityId}/reviews	Activity reviews and ratings.
Lookups	GET /api/Categories; GET /api/Cities	Dynamic filters and profile preferences.
Wishlist	GET /api/wishlist; GET /api/wishlist/activity-ids; POST /api/wishlist/{activityId}; DELETE /api/wishlist/{activityId}	Saved activities.
Plan	GET /api/plan; POST /api/plan/items; DELETE /api/plan/items/{planItemId}; GET /api/plan/activity-ids	Itinerary schedule.
Profile	GET/PUT /api/users/me/profile; POST/DELETE /api/users/me/profile-picture	Profile data and picture.
Admin activities	GET/POST/PUT/DELETE /api/admin/activities	Admin catalogue management.
Admin users	GET/POST/PUT/DELETE /api/users	Admin user management.
Dashboard/Reports	GET /api/dashboard; reports endpoints	Admin metrics and charts.
Chat conversations	GET/POST/DELETE /api/chat/conversations	Chat history metadata.

Table 10. Backend API modules

5.4 Security and Authorization

Security is implemented using JWT Bearer authentication. Protected endpoints require a valid token, and admin-only functions require the Admin role. The frontend includes guards to prevent unauthorized navigation,

but the backend remains the source of truth for authorization. Password reset is implemented through email-based verification, and SQL access is handled by EF Core rather than manual string concatenation.

5.5 Image Handling

Profile pictures and activity images are uploaded as multipart/form-data. The backend sends the image to Cloudinary and stores the returned URL in the database. This keeps the database smaller and avoids serving large binary files directly from SQL Server.

5.6 Controller Responsibilities

Controller	Responsibility
AuthController	Handles register, login, forgot password, reset-code verification and password reset requests.
ActivitiesController	Serves the public activity catalogue, filterable cards, activity details and activity reviews.
AdminActivitiesController	Provides admin-only create, update and delete operations for activities and related images, hours and booking links.
UsersController	Supports admin user listing, filtering, details and user-management operations.
UserProfilesController	Serves the current tourist profile and profile-picture operations.
WishlistController	Handles saved activities and the lightweight activity-id list used by the frontend hearts.
PlanController	Handles scheduled activities and plan item removal.
DashboardController / ReportsController	Returns admin statistics, report cards and chart data.
ChatConversationsController	Stores and retrieves conversation metadata for chatbot history.

Table 11. Controller responsibilities

5.7 Data Transfer Objects

DTOs are used so the frontend does not depend directly on database entity classes. For example, the activity card response is smaller than the activity details response. Admin requests also use dedicated DTOs because activity create/update needs extra data such as images, opening hours and booking links.

DTO group	Use
ActivityCardDto	Compact data for activity cards in list, wishlist and related views.
ActivityDetailsDto	Full details page data including gallery, hours, documents, reviews and meeting point.
AuthDtos	Login/register payloads and authentication response.
PlanDtos / WishlistDtos	Payloads and responses for schedule and saved activities.
UserProfileDtos	Profile form data, preferred cities/categories and travel dates.
ReportDtos / DashboardDto	Admin metrics and chart data.

Table 12. DTO usage summary

5.8 Core Backend Flows

Flow	Backend processing
Activity listing	Frontend sends filters -> ActivitiesController -> ActivityService builds query -> EF Core reads activities with category/city/images -> DTOs returned.
Activity details	Route id is sent -> details query loads one activity with related data -> details DTO returned to Angular.
Wishlist add/remove	JWT identifies current user -> activity id is added or removed from WishlistItems -> updated list/id status is returned.
Schedule item	JWT identifies current user -> PlanItem is created with selected activity/date/time -> schedule page reloads itinerary.
Profile update	Current user profile is loaded -> updated profile fields and preferences are saved -> frontend refreshes profile state.
Admin activity update	Admin sends multipart/form-data -> backend validates and maps fields -> images/links/hours are updated -> admin table reloads.

Table 13. Core backend processing flows

Chapter 6 - Frontend Implementation

6.1 Frontend Structure

The frontend is an Angular Standalone application using feature-based organization. Components are standalone, routes are defined in Angular route files, and services handle API communication. Tailwind CSS and Flowbite provide the visual structure, while project-specific components such as Navbar, Footer, Chatbot and Dashboard layout make the interface reusable.

Frontend area	Details
Core components	Navbar, footer, layout components and chatbot.
Auth feature	Login, register, forgot password and reset password pages.
Tourist features	Home, activities, details, wishlist, schedule, map, profile and contact.
Admin feature	Dashboard shell, users management, activities management and reports.
Core services	Auth interceptor, cache interceptor, guards, constants and shared services.
UI libraries	Tailwind CSS, Flowbite, Font Awesome, ngx-toastr, ngx-spinner, jsPDF, ApexCharts.

Table 14. Frontend code organization

6.2 Angular Routes

Route	Screen
/home	Landing page and public introduction.
/auth/login	Login screen.
/auth/register	Registration screen.
/auth/forgot-password	Request reset code.
/auth/reset-password	Create new password after verification.
/activities	Activities listing and filters.
/activities/:id	Dynamic activity details route.
/wishlist	Saved activities.
/schedule	Trip schedule.
/map	Map view of scheduled items.
/profile	User profile.
/contact	Support/contact form.
/dashboard	Admin dashboard shell.
/dashboard/users-management	Admin users.
/dashboard/activities-management	Admin activities.
/dashboard/reports	Admin reports.

Table 15. Final frontend routes

6.3 API Integration from Frontend

- AuthService communicates with register, login and password-reset endpoints.
- ActivitiesService loads cards, details, categories and cities.
- WishlistService loads saved activities, wishlist IDs and add/remove operations.
- ScheduleService manages plan items and exports schedule information.
- ProfileService loads and updates profile data and profile pictures.
- Dashboard/admin services connect the admin layout to user, activity and report APIs.
- ChatbotService connects the user/admin chatbot UI to the AI RAG service and conversation flow.

6.4 UI/UX Decisions

The interface uses a dark visual identity with gold/yellow accents. Tourist pages focus on discovery and planning, while admin pages use tables, filters and action buttons. The final UI also includes empty/error states, loading behavior, profile initials fallback, image previews and export actions. The design is responsive and uses cards and sections to make complex data readable.

6.5 Component Responsibilities

Component	Responsibility
NavbarComponent	Displays navigation, login/profile state and dynamic avatar/initials.
HomeComponent	Presents the platform, city sections, popular activities and entry points.
ActivitiesComponent	Loads activity cards, filters, search, sorting, wishlist state and add-to-plan actions.
DetailsComponent	Reads the route id, loads one activity, displays gallery, reviews, documents and meeting point.
WishlistComponent	Displays saved cards and allows remove/add-to-plan/view-details actions.
ScheduleComponent	Displays itinerary items, totals and export actions.
MapComponent	Displays Google map markers and itinerary side panel.
ProfileComponent	Loads and saves profile fields, travel dates, preferred cities/categories and image state.
Admin Dashboard Components	Provide admin shell, cards, users table, activities table and report charts.
ChatbotComponent	Provides floating chat panel, conversation history and message input.

Table 16. Frontend component responsibilities

6.6 State and Error Handling

The frontend uses feature-level state variables and services to control loading, filtering, pagination, selected items, form values and modal visibility. Errors are displayed through user-facing messages instead of failing silently. Protected pages depend on the token and role stored after login, while SSR-sensitive code avoids direct browser API access when the app is rendered on the server.

- Loading states prevent the user from interacting with incomplete data.
- Filter state in activities and admin modules is separated from API services for clarity.
- Profile and navbar avatar logic uses uploaded image when available and initials as fallback.
- Admin CRUD modals separate view, add, update and delete confirmation actions.
- The screenshot automation helped validate that real user/admin sessions can open the expected routes.

Chapter 7 - AI/RAG Implementation

7.1 AI Module Overview

The AI module is implemented as a separate FastAPI service. Its final role is not a generic AI recommender; it is a RAG knowledge-base assistant. Administrators upload documents, the service extracts text, splits it into chunks, creates embeddings and stores vector data so that user questions can be answered from the uploaded knowledge base.

RAG File Processing and Chat Flow

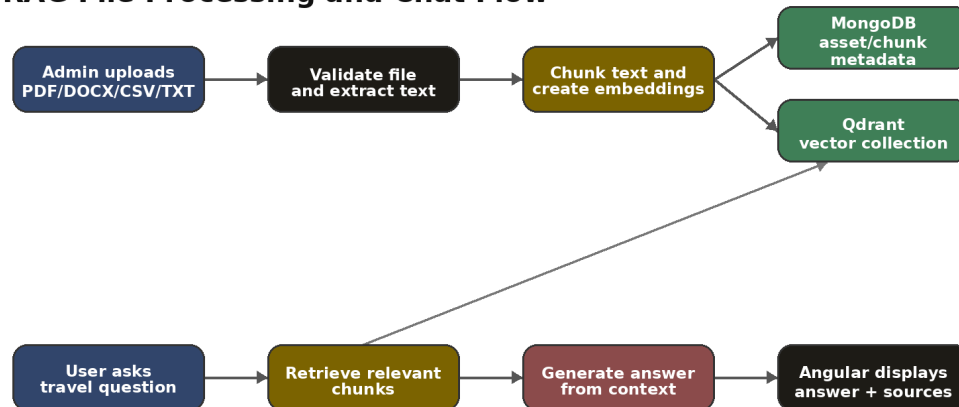


Figure 7. RAG upload, indexing and chat-answer flow

7.2 AI/RAG Endpoints

Endpoint	Purpose
GET /	Health/basic service response.
POST /api/v1/data/multi-upload	Upload one or more files to be indexed.
GET /api/v1/data/files	Return uploaded/indexed files.
PUT /api/v1/data/file/{file_id}	Update a file in the knowledge base.
DELETE /api/v1/data/file/{file_id}	Delete a file and its related indexed data.
POST /api/v1/nlp/chat	Receive a question and return an answer from retrieved context.

Table 17. FastAPI RAG endpoints

7.3 RAG Storage

- MongoDB stores file and chunk metadata because document information is flexible and document-oriented.
- Qdrant stores embeddings and supports semantic similarity search over uploaded content.
- The LLM provider generates the final answer using retrieved context rather than relying only on its general knowledge.
- Supported file types include PDF, DOCX, CSV and TXT, which makes the admin panel practical for tourism content updates.

7.4 Why the AI Service Is Separate

Separating the AI service from the .NET backend makes the system easier to deploy and maintain. The AI service has Python dependencies, embedding/vector database requirements and different scaling needs. Keeping it separate avoids mixing those concerns with the main business API.

7.5 AI Service Code Organization

AI file/module	Responsibility
routes/data.py	Defines file upload/list/update/delete routes.
routes/nlp.py	Defines chat route used by the chatbot.
controllers/DataController.py	Coordinates file validation, storage and processing.
controllers/NLPController.py	Coordinates retrieval and answer generation.
models/AssetModel.py and ChunkModel.py	Represent uploaded assets and extracted chunks.
stores/llm/providers/OpenAIProvider.py	Connects to the LLM/embedding provider.
stores/vectordb/providers/QdrantDBProvider.py	Handles vector indexing and similarity search.
assets/files	Contains uploaded tourism knowledge documents such as Cairo, Alexandria, Luxor, Aswan and Egypt overview PDFs.

Table 18. AI/RAG code organization

7.6 AI Answering Behavior

When a user sends a message, the service embeds the question, retrieves the most relevant chunks from Qdrant, prepares a context prompt and asks the language model to answer using the retrieved content. This approach reduces hallucination compared with a pure generic chatbot because the answer is grounded in project-controlled documents.

Chapter 8 - Graphical User Interface and Feature Walkthrough

This chapter documents the final visible screens. Each subsection shows the screenshot first, followed by a compact explanation of what the user sees, what actions can be performed and what system behavior is involved. This order makes the section easier to review during the project discussion.

8.1 Home Page

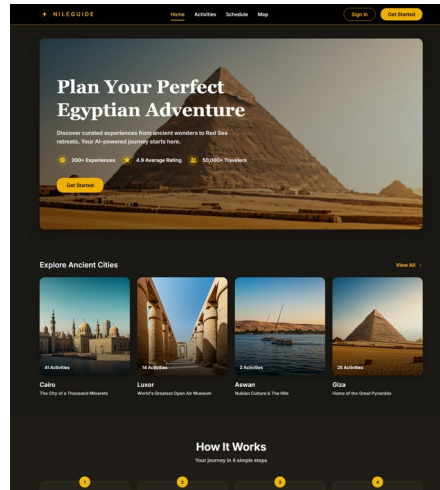


Figure 8. Home Page

Purpose	Introduce NileGuide and guide visitors into the main tourist journey.
User/Admin actions	The visitor can read the platform value, use the navigation bar, move to activities, login/register, view city highlights and popular activities.
System behavior	The page is a public Angular route. It uses static marketing sections and activity/city presentation components to direct users to the discovery flow.
Implementation status	Implemented public screen.

8.2 Login Page

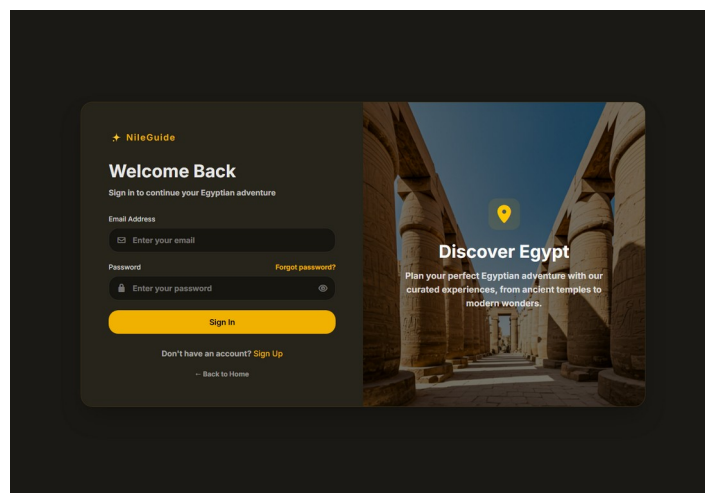


Figure 9. Login Page

Purpose	Authenticate existing tourists or administrators.
User/Admin actions	The user enters email and password, then submits the form. Navigation links allow account creation or password recovery.
System behavior	The frontend calls the login endpoint and stores the returned JWT token, role and user id. Route guards then allow tourist or admin access depending on role.
Implementation status	Implemented. Remember Me is not treated as a delivered feature.

8.5 Activity Details Page

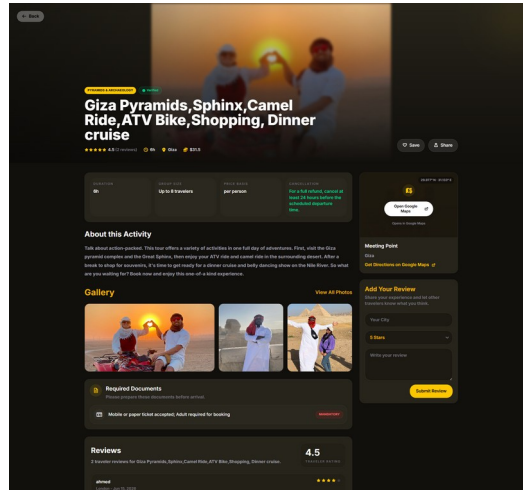


Figure 12. Activity Details Page

Purpose	Show complete information about one selected activity.
User/Admin actions	The user can review the hero section, Save/Share actions, duration, group size, price, cancellation, about section, gallery, required documents, reviews, meeting point and add-review form.
System behavior	The page is opened through /activities/:id and loads one activity from the backend. This proves that the details page is dynamic rather than static.
Implementation status	Implemented dynamic details screen.

8.6 Wishlist Page

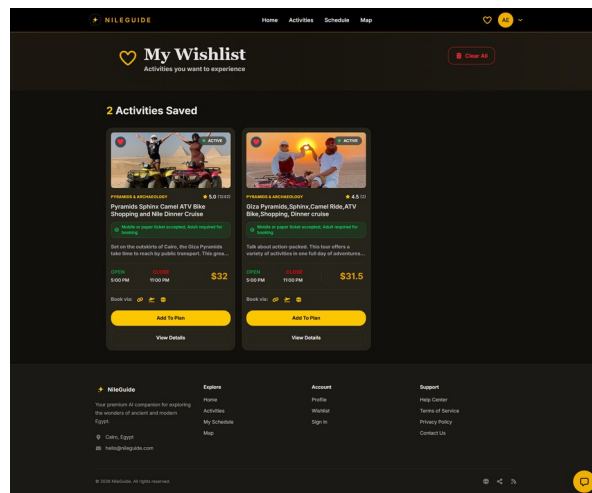


Figure 13. Wishlist Page

Purpose	Keep activities the tourist is interested in before scheduling.
User/Admin actions	The tourist can review saved activity cards, clear all saved items, remove individual activities, add to plan or view details.
System behavior	The frontend calls wishlist endpoints to load saved cards and wishlist activity ids. Wishlist is separate from schedule, so saving does not automatically plan the activity.
Implementation status	Implemented.

8.7 Schedule Page

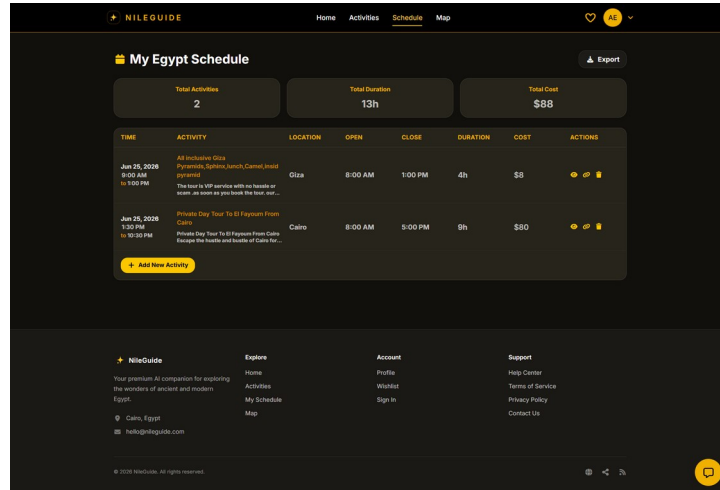


Figure 14. Schedule Page

Purpose	Display the tourist itinerary as a structured plan.
User/Admin actions	The tourist can see number of scheduled activities, total duration, estimated cost, and planned activities by date. Export/print actions support taking the plan outside the website.
System behavior	Plan items are loaded from the backend. The frontend calculates summary values and can generate a PDF/export view using client-side tools.
Implementation status	Implemented.

8.8 Map Page

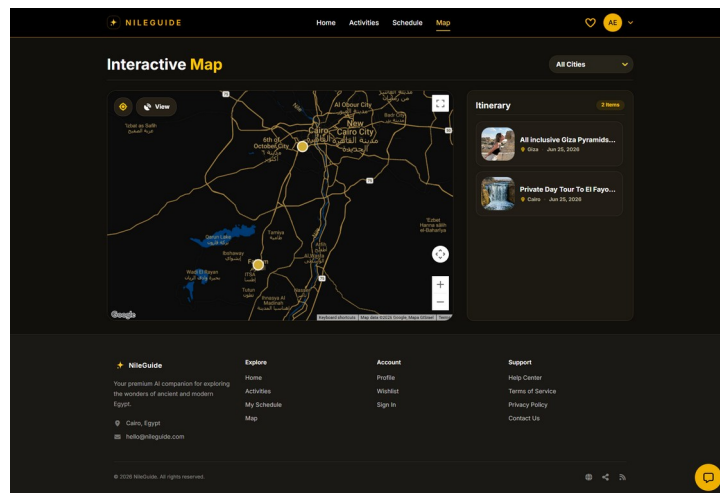


Figure 15. Map Page

Purpose	Show scheduled activities geographically.
User/Admin actions	The tourist can view map markers, see the itinerary list on the side and filter planned activities by city.
System behavior	The map page uses activity coordinates and scheduled plan data. It links the planning feature to a visual location view.
Implementation status	Implemented.

8.9 Profile Page

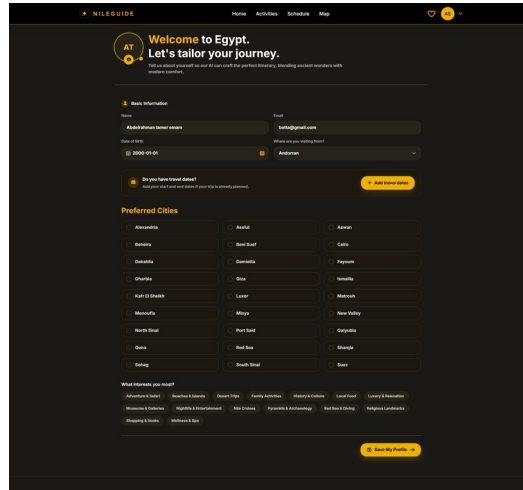


Figure 16. Profile Page

Purpose	Manage personal information and travel preferences.
User/Admin actions	The user can view initials/profile picture, edit basic information, travel dates, preferred cities and interest categories, then save the profile.
System behavior	Profile data comes from /users/me/profile and updates are sent back to the backend. Selected cities/categories can later help filtering and recommendations.
Implementation status	Implemented; budget is not shown in the final profile screen.

8.10 User Chatbot Panel

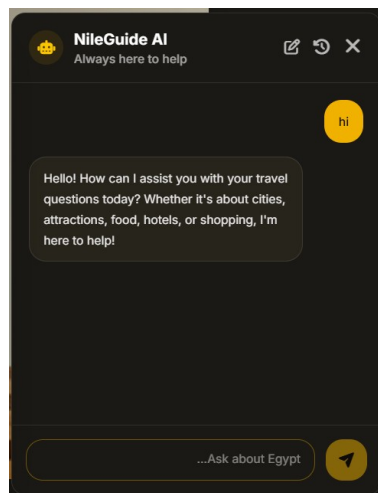


Figure 17. User Chatbot Panel

Purpose	Allow tourists to ask NileGuide questions from the website.
User/Admin actions	The user opens the floating chatbot, sees the assistant greeting, writes a travel question and can start or access chats.
System behavior	The frontend sends the prompt to the FastAPI RAG endpoint. The answer is generated from retrieved knowledge chunks when relevant content is available.
Implementation status	Implemented RAG chatbot UI.

8.11 Chat History Panel

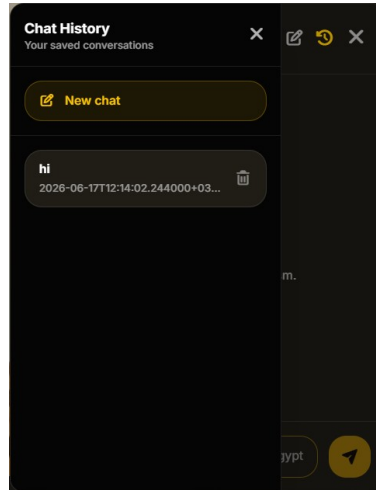


Figure 18. Chat History Panel

Purpose	Show previous chatbot conversations.
User/Admin actions	The user can open a previous conversation, start a new chat or delete an old chat from the history list.
System behavior	Conversation metadata is stored separately from the RAG answer generation flow, allowing the UI to reopen earlier chats.
Implementation status	Implemented conversation-history UI.

8.12 Admin Dashboard and Users Management

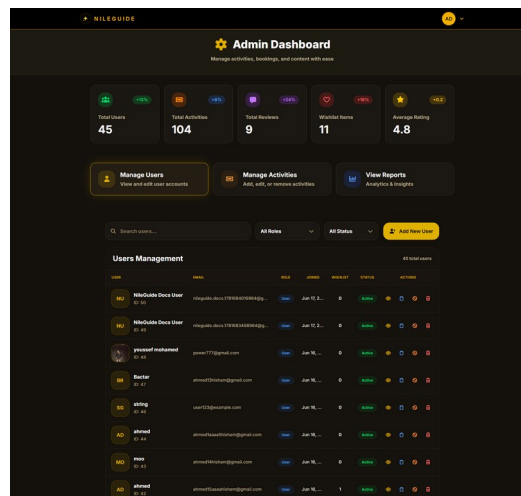


Figure 19. Admin Dashboard and Users Management

Purpose	Give administrators an overview and tools for managing users.
User/Admin actions	The admin sees metric cards, searches users, filters by role/status, adds new users and uses action icons for each row.
System behavior	This page is protected by admin role. It combines dashboard metrics with a users table loaded from backend admin/user endpoints.
Implementation status	Implemented admin module.

8.13 Admin Activities Management

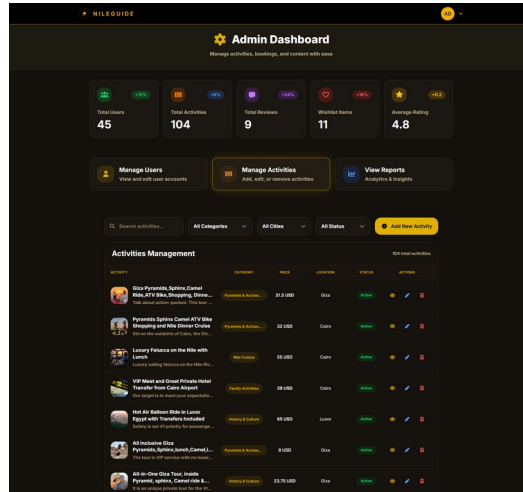


Figure 20. Admin Activities Management

Purpose	Maintain the activity catalogue shown to tourists.
User/Admin actions	The admin can search, filter by category/city/status, add a new activity and use view/edit/delete actions on activity rows.
System behavior	Admin activity operations use backend admin endpoints. Activity images are handled through multipart/form-data and image URLs are stored after upload.
Implementation status	Implemented admin module.

8.14 Admin Reports Page

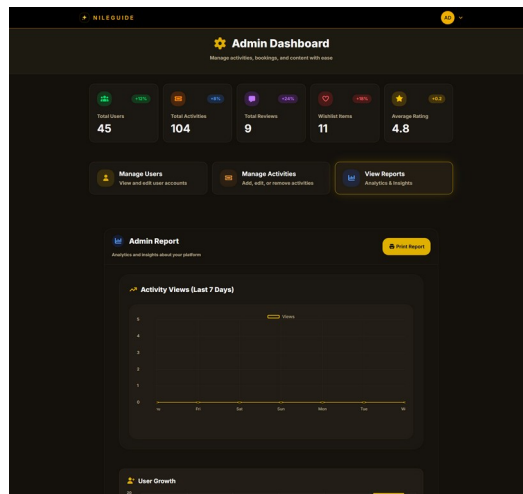


Figure 21. Admin Reports Page

Purpose	Present analytics for platform monitoring.
User/Admin actions	The admin can view summary cards, print the report and inspect charts such as activity views and user growth.
System behavior	Report data comes from backend reporting endpoints. During testing, charts may reflect the available seeded/demo data.
Implementation status	Implemented report screen.

8.15 Admin AI Files Panel

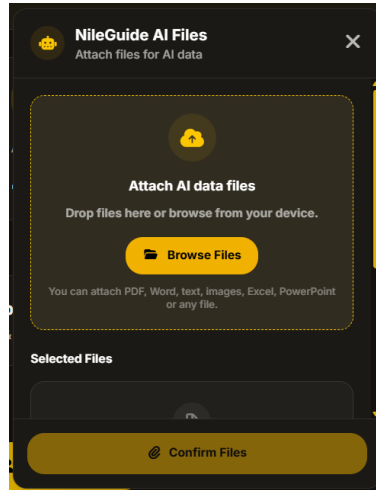


Figure 22. Admin AI Files Panel

Purpose	Maintain the documents used by the AI assistant.
User/Admin actions	The admin can open the AI files panel, drag or browse files, attach them and submit them to the knowledge base.
System behavior	Uploaded files are sent to the FastAPI RAG service, which extracts text, creates chunks and indexes embeddings in Qdrant while storing metadata in MongoDB.
Implementation status	Implemented upload panel; list/update/delete are supported by the AI API.

8.16 Contact Page

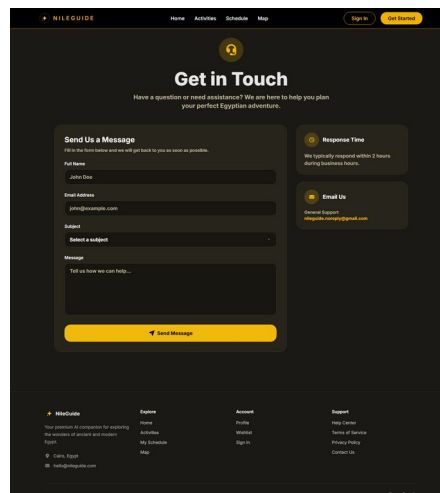


Figure 23. Contact Page

Purpose	Provide a public support channel.
User/Admin actions	The user enters name, email, subject and message. Support details and the official email are displayed on the right.
System behavior	The contact form uses the frontend contact implementation and configured email/contact behavior. The official support email is nileguide.noreply@gmail.com.
Implementation status	Implemented public support screen.

8.17 Forgot Password Page

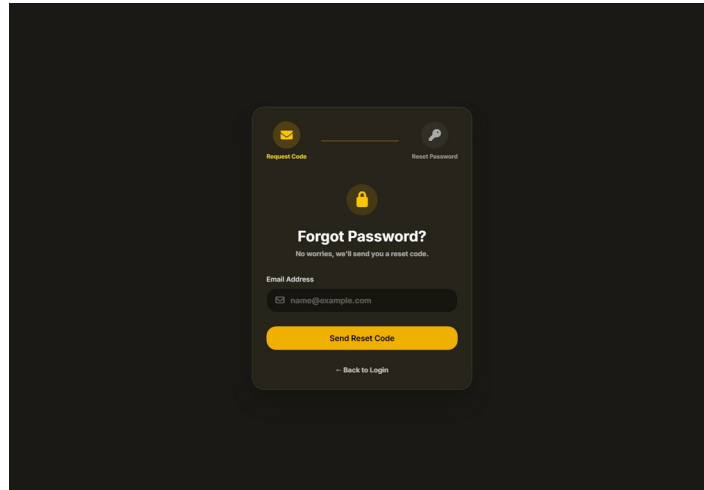


Figure 24. Forgot Password Page

Purpose	Start the password recovery process.
User/Admin actions	The user enters email to receive a reset code.
System behavior	The frontend calls the forgot-password endpoint. The backend sends or prepares the verification code through the configured email service.
Implementation status	Implemented.

8.18 Reset Password Page

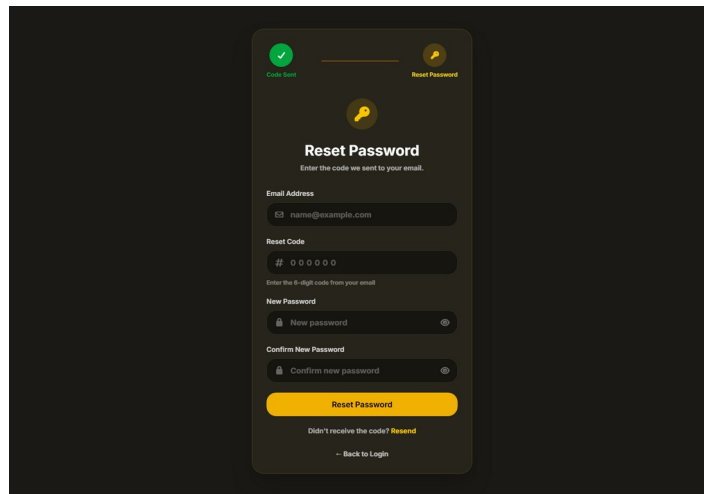


Figure 25. Reset Password Page

Purpose	Complete password recovery after verification.
User/Admin actions	The user enters email, reset code, new password and confirmation.
System behavior	The backend verifies the reset code and updates the password after validation. The user can then return to login.
Implementation status	Implemented.

Chapter 9 - Testing and Validation

9.1 Testing Strategy

Testing focused on validating the main flows that would be demonstrated during the graduation discussion. The project used manual browser testing, API testing through Swagger/Postman-style calls, build checks, and automated screenshot capture to confirm that pages render and protected routes can be reached using real tourist/admin accounts.

Test area	What was checked	Result
Public navigation	Home, login, register, forgot password, reset password and contact pages.	Passed through screenshot run.
Tourist login	Login as tourist and access activities, wishlist, schedule, map, profile and chatbot.	Passed through API login and screenshots.
Admin login	Login as admin and access dashboard modules.	Passed through API login and screenshots.
Activity details	Open selected activity by /activities/:id and show dynamic content.	Passed after direct route capture.
Wishlist	Saved activities are shown and user can remove/add to plan.	Validated visually and through protected route access.
Schedule/Map	Planned activities are presented as itinerary and map markers.	Validated visually.
RAG upload panel	Admin can open AI file panel and attach documents.	Validated visually; API supports file operations.

Table 19. Testing summary

9.2 Common Issues Fixed During Development

- Protected pages sometimes redirected to login until token storage and role keys were aligned between login and guards.
- SSR/localStorage errors were avoided by checking browser platform before accessing browser-only APIs.
- Activity details route was finalized as /activities/:id instead of a static Details page.
- Activity image update/create flows used FormData instead of only URL fields.
- Profile picture fallback initials were added to avoid broken images when the user has no uploaded photo.
- Gemini/frontend API-key usage was removed from the frontend scope; AI operations are handled through backend/AI services.

9.3 Validation Against Scope

Scope item	Final validation
Discovery and planning	Delivered through activities, details, wishlist, schedule and map.
Admin operations	Delivered through users, activities, reports and AI file modules.
AI knowledge assistant	Delivered as FastAPI RAG service connected to chatbot and admin upload panel.
Recommendation engine	Partially addressed through AI knowledge answers; full personalized recommender remains future work.
Booking/payment	Not included in the final delivered release.

Table 20. Scope validation

Chapter 10 - Conclusion and Future Work

10.1 Conclusion

NileGuide was delivered as a complete web-based tourism planning platform with a working frontend, backend, database and AI/RAG module. The final system supports tourists in discovering and planning activities and supports administrators in maintaining users, activities, reports and AI knowledge files. The implementation is not only a static website; it includes real API integration, protected routes, dynamic activity data, image handling, schedule/wishlist persistence and a separate AI document-processing service.

The project also shows a clear evolution from the midyear plan. Some ideas were refined or postponed, while the core platform became stronger. Instead of documenting everything as if it was delivered, the final report separates implemented features from future work. This makes the project easier to defend because the documentation matches the actual code and screenshots.

10.2 Limitations

- The system does not include real payment or booking confirmation with external providers.
- The AI assistant depends on the quality and coverage of uploaded knowledge files.
- Advanced personalized recommendation based on budget, travel dates and user preferences is not fully implemented.
- Activity-view analytics depend on available tracking data; richer reports require consistent event logging.
- Refresh-token and Remember Me user flows are not documented as delivered features.

10.3 Future Work

- Add a full recommendation engine using user preferences, cities, interests, ratings and schedule constraints.
- Add real booking-provider integration after selecting stable provider APIs and defining the business process.
- Improve schedule optimization by detecting conflicts and considering travel time between activities.
- Enhance AI answers with better source display, document tags and admin review of generated responses.
- Add Arabic interface support and multilingual chatbot responses.
- Expand analytics by logging activity views, conversion events, wishlist trends and schedule additions.
- Implement secure long-session handling if the product later requires Remember Me or refresh-token behavior.

Chapter 11 - References

- [1] Angular Documentation. <https://angular.dev/>
- [2] Microsoft ASP.NET Core Documentation. <https://learn.microsoft.com/aspnet/core/>
- [3] Microsoft Entity Framework Core Documentation. <https://learn.microsoft.com/ef/core/>
- [4] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [5] Qdrant Documentation. <https://qdrant.tech/documentation/>
- [6] MongoDB Documentation. <https://www.mongodb.com/docs/>
- [7] Cloudinary Documentation. <https://cloudinary.com/documentation>
- [8] Chang, H.-T., Chang, Y.-M., & Tsai, M.-T. (2016). ATIPS: Automatic Travel Itinerary Planning System for Domestic Areas. Computational Intelligence and Neuroscience.
- [9] Hassannia, R., Vatankhah Barenji, A., Li, Z., & Alipour, H. (2019). Web-Based Recommendation System for Smart Tourism: Multiagent Technology. Sustainability, 11(2), 323.
- [10] Srisawatsakul, C., & Boontarig, W. (2020). Tourism Recommender System using Machine Learning Based on User Public Instagram Photos. IEEE InCIT 2020.
- [11] Kumar, K., Birla, V., Sasi, A., & Deep, A. (2020). Design and Development of Travel and Tourism Recommendation System using Web-Scraped Data Positioned on Artificial Intelligence and Machine Learning. International Journal of Advanced Trends in Computer Science and Engineering, 9(4), 5670-5679.

Chapter 12 - Appendices

Appendix A - Final Feature Checklist

Area	Items	Status
Public pages	Home, login, register, forgot password, reset password, contact	Implemented
Tourist planning	Activities, details, wishlist, schedule, map, profile, chatbot	Implemented
Admin dashboard	Users, activities, reports, AI file panel	Implemented
AI/RAG	Upload/list/update/delete files and chat answers from retrieved context	Implemented
Real booking/payment	Provider booking confirmation and payment processing	Future work
Remember Me / refresh-token UX	Long-session user experience	Future work

Table 21. Final feature checklist

Appendix B - Source Code Structure

Part	Important code/assets
Frontend	src/app/core, src/app/features, app.routes.ts, feature components, services, guards and interceptors.
Backend	Controllers, DTOs, Models, Services, Data/AppDbContext, Options, Middleware and Hubs.
AI/RAG	FastAPI routes, controllers, models, LLM provider, Qdrant provider, MongoDB models and uploaded file assets.
Documentation assets	Screenshots, diagrams, ERD, workflow images and the official EELU template/logo.

Table 22. Source code organization appendix